Rain BitTorrent 클라이언트 시스템 아키텍처 개요

목차

1. 개요
2. 전체 시스템 구조
3. 계층별 아키텍처
4. 핵심 컴포넌트
5. Session과 Torrent의 관계
6. 패키지 구조 분석

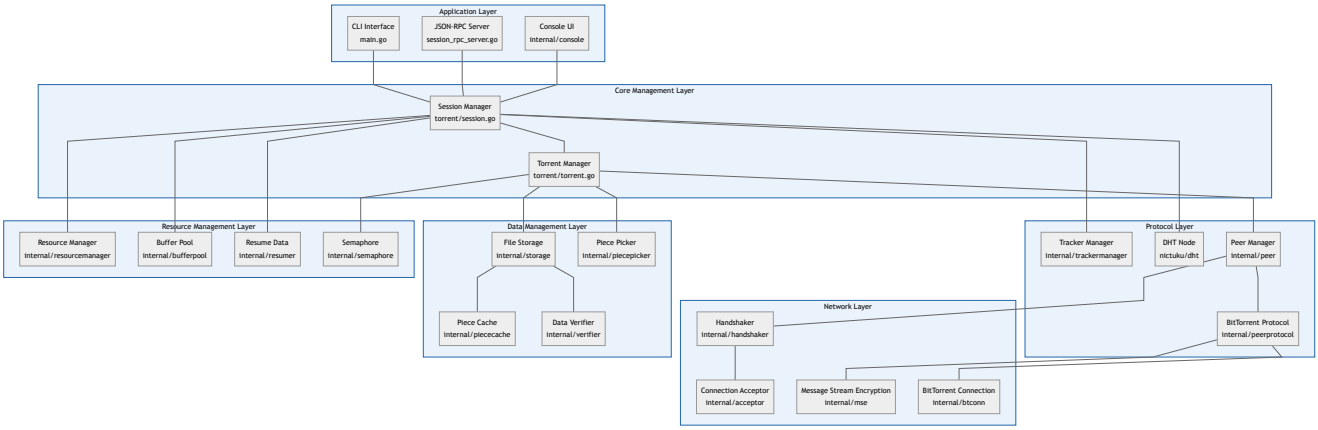
개요

Rain BitTorrent 클라이언트는 Go 언어로 구현된 고성능 BitTorrent 클라이언트로, 서버 환경에서의 대규모 토렌트 처리에 최적화되어 있습니다. 이 문서는 Rain의 전체 시스템 아키텍처를 분석하여 각 컴포넌트의 역할과 상호작용을 설명합니다.

주요 특징

- **서버 우선 설계**: 수백 개의 동시 토렌트 처리 가능
- **토렌트별 개별 포트**: 프라이빗 트래커 지원을 위한 독립적인 네트워크 스택
- **JSON-RPC API**: 원격 관리 및 제어 인터페이스
- **지속성 지원**: BoltDB를 통한 재시작 데이터 보존
- **확장 프로토콜**: DHT, PEX, MSE, WebSeed 등 다양한 BitTorrent 확장 지원

전체 시스템 구조


Application Layer
CLI Interface
main.go
JSON-RPC Server
session_rpc_server.go
Console UI
internal/console
Core Management Layer
Session Manager
torrent/session.go
Torrent Manager
torrent/torrent.go
Resource Management Layer
Resource Manager
internal/resourcemanager
Buffer Pool
internal/bufferpool
Resume Data
internal/resumer
Semaphore
internal/semaphore
Data Management Layer
File Storage
internal/storage
Piece Picker
internal/piecepicker
Piece Cache
internal/piececache
Data Verifier
internal/verifier
Protocol Layer
Tracker Manager
internal/trackermanager
DHT Node
nictuku/dht
Peer Manager
internal/peer
BitTorrent Protocol
internal/peerprotocol
Network Layer
Handshaker
internal/handshaker
Connection Acceptor
internal/acceptor
Message Stream Encryption
internal/mse
BitTorrent Connection
internal/btconn

계층별 아키텍처

1. Application Layer (애플리케이션 계층)

사용자 인터페이스와 외부 API를 제공하는 최상위 계층입니다.

구성 요소:

- **CLI Interface** (`main.go`): 명령줄 인터페이스
- **JSON-RPC Server** (`session_rpc_server.go`): 원격 API 서버
- **Console UI** (`internal/console`): 실시간 모니터링 콘솔

책임:

- 사용자 명령 처리
- 원격 클라이언트 요청 처리
- 실시간 상태 표시

2. Core Management Layer (핵심 관리 계층)

전체 시스템의 생명주기와 토렌트 관리를 담당합니다.

구성 요소:

- **Session Manager** (`torrent/session.go`): 전체 클라이언트 세션 관리
- **Torrent Manager** (`torrent/torrent.go`): 개별 토렌트 관리

책임:

- 토렌트 추가/제거/시작/중지
- 전역 설정 및 리소스 관리
- 토렌트 간 조정

3. Protocol Layer (프로토콜 계층)

BitTorrent 프로토콜과 확장 프로토콜을 구현합니다.

구성 요소:

- **Tracker Manager** (`internal/trackermanager`): 트래커 통신 관리
- **DHT Node** (`nictuku/dht`): 분산 해시 테이블
- **Peer Manager** (`internal/peer`): 피어 상태 관리
- **BitTorrent Protocol** (`internal/peerprotocol`): 프로토콜 메시지 처리

책임:

- 트래커와의 통신
- DHT를 통한 피어 발견
- 피어 간 프로토콜 메시지 교환

4. Network Layer (네트워크 계층)

네트워크 연결과 암호화를 처리합니다.

구성 요소:

- **Connection Acceptor** (`internal/acceptor`): 인커밍 연결 수락
- **Handshaker** (`internal/handshaker`): 핸드셰이크 처리
- **MSE** (`internal/mse`): 메시지 스트림 암호화
- **BitTorrent Connection** (`internal/btconn`): BitTorrent 연결 추상화

책임:

- TCP 연결 관리
- 핸드셰이크 프로토콜 수행
- 연결 암호화

5. Data Management Layer (데이터 관리 계층)

파일 저장, 피스 관리, 데이터 검증을 담당합니다.

구성 요소:

- **File Storage** (`internal/storage`): 파일 시스템 인터페이스
- **Piece Cache** (`internal/piececache`): 피스 캐싱 시스템
- **Piece Picker** (`internal/piecepicker`): 피스 선택 알고리즘
- **Data Verifier** (`internal/verifier`): 데이터 무결성 검증

책임:

- 파일 I/O 처리
- 피스 선택 전략
- 데이터 검증 및 캐싱

6. Resource Management Layer (리소스 관리 계층)

시스템 리소스와 동시성을 관리합니다.

구성 요소:

- **Resource Manager** (`internal/resourcemanager`): 연결 수 제한
- **Buffer Pool** (`internal/bufferpool`): 메모리 풀 관리
- **Resume Data** (`internal/resumer`): 재시작 데이터 관리
- **Semaphore** (`internal/semaphore`): 동시성 제어

책임:

- 메모리 및 연결 리소스 제한
- 재시작 데이터 지속성
- 동시성 제어

핵심 컴포넌트

Session (세션)

```go
type Session struct {
config Config
db *bbolt.DB
storage storage.Provider
resumer *boltdbresumer.Resumer
dht *dht.DHT
rpc *rpcServer
trackerManager *trackermanager.TrackerManager
ram *resourcemanager.ResourceManager[*peer.Peer]
pieceCache *piececache.Cache
torrents  map[string]*Torrent
// ...
}
```

역할:

- 전체 BitTorrent 클라이언트의 최상위 컨테이너
- 여러 토렌트를 관리하는 중앙 집중식 관리자
- 공유 리소스 (DHT, 캐시, 데이터베이스) 관리

주요 기능:

- `NewSession(cfg Config)` : 새 세션 생성 및 초기화

- `AddTorrent()`, `AddURI()` : 토렌트 추가
- `ListTorrents()` : 토렌트 목록 조회
- `Close()` : 세션 종료 및 리소스 정리

Torrent (토렌트)

```go
type torrent struct {
session *Session
id  string
infoHash [20]byte
trackers []tracker.Tracker
storage storage.Storage
port  int
info *metainfo.Info
bitfield *bitfield.Bitfield
peers  map[*peer.Peer]struct{}
piecePicker *piecepicker.PiecePicker
// ...
}
```

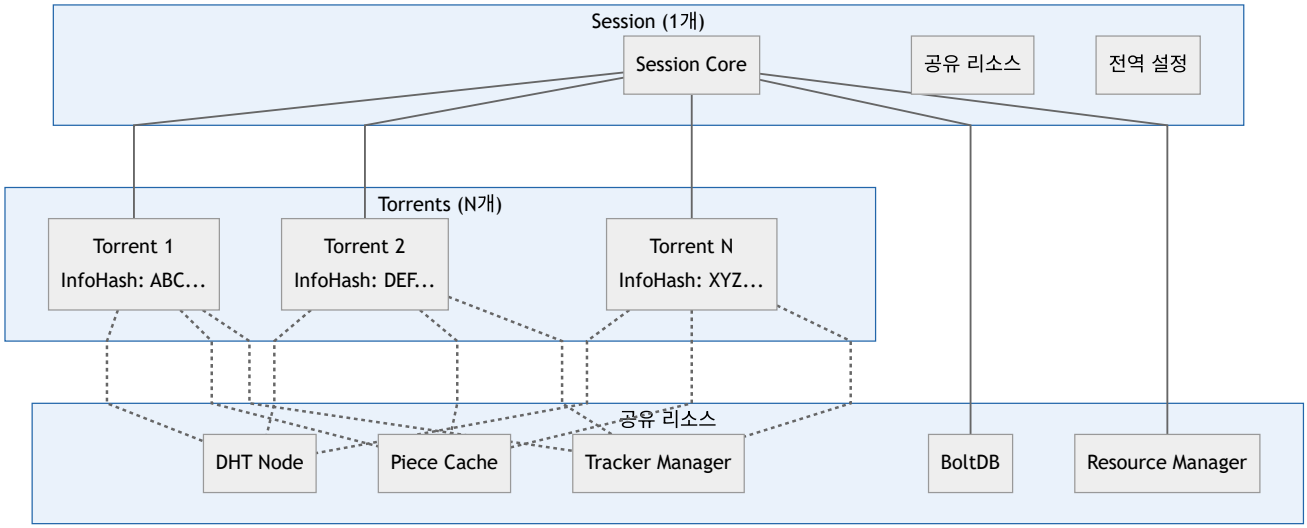
역할:

- 개별 토렌트의 다운로드/업로드 관리
- 피어 연결 및 데이터 교환 조정
- 토렌트별 상태 및 통계 관리

주요 기능:

- `Start()`, `Stop()`: 토렌트 시작/중지
- `run()`: 메인 토렌트 이벤트 루프
- `Stats()`: 토렌트 통계 정보 제공

Session과 Torrent의 관계



관계 특성

1. **1:N 관계**: 하나의 Session이 여러 Torrent를 관리
2. **리소스 공유**: 모든 토렌트가 Session의 공유 리소스 사용
3. **독립적 포트**: 각 토렌트는 고유한 TCP 포트 할당
4. **통합 관리**: Session이 토렌트 생명주기 전체 관리

데이터 흐름

1. **토렌트 추가**: `Session.AddTorrent()` → 새 `torrent` 인스턴스 생성
2. **리소스 할당**: Session이 포트, 저장소, 트래커 할당
3. **독립 실행**: 각 토렌트가 독립적인 고루틴에서 실행
4. **상태 동기화**: Session이 모든 토렌트 상태 추적 및 관리

패키지 구조 분석

Rain의 internal 패키지는 기능별로 명확히 분리되어 있습니다:

네트워킹 관련

- `acceptor` : 인커밍 연결 수락
- `handshaker` : 피어 핸드셰이크 처리
- `btconn` : BitTorrent 연결 추상화
- `mse` : 메시지 스트림 암호화
- `peerconn` : 피어 연결 관리

데이터 관리

- `piece` : 피스 상태 관리
- `bitfield` : 비트필드 구현
- `storage` : 파일 저장 인터페이스
- `piececache` : 피스 캐싱
- `verifier` : 데이터 검증

프로토콜 구현

- `peerprotocol` : BitTorrent 프로토콜 메시지

- `tracker` : 트래커 통신
- `announcer` : 주기적 announce
- `infodownloader` : 메타데이터 다운로드

알고리즘 및 전략

- `piecepicker` : 피스 선택 알고리즘
- `piecedownloader` : 피스 다운로드 관리
- `unchoker` : Choking/Unchoking 알고리즘
- `peerpriority` : 피어 우선순위

리소스 관리

- `resourcemanager` : 연결 수 제한
- `bufferpool` : 메모리 풀
- `semaphore` : 동시성 제어
- `allocator` : 파일 할당

유틸리티

- `logger` : 로깅 시스템
- `metainfo` : 토렌트 파일 파싱
- `magnet` : 마그넷 링크 처리
- `resolver` : DNS 해석

이러한 모듈화된 구조는 각 기능의 독립적인 개발과 테스트를 가능하게 하며, 코드의 재사용성과 유지보수성을 크게 향상시킵니다.

관련 문서

선행 지식

- 00-index.md - 전체 문서 구조와 읽기 가이드

관련 주제

- 02-data-flow-analysis.md - 컴포넌트 간 데이터 흐름 상세 분석
- 04-01-networking-packages-analysis.md - 네트워킹 계층 구현 세부사항
- 06-01-performance-optimization-analysis.md - 아키텍처 기반 성능 최적화

후속 학습

- 03-01-torrent-lifecycle-process.md - 토렌트 라이프사이클 상세 분석
- 05-01-dht-implementation-analysis.md - DHT 구현 세부사항
- 07-01-library-usage-guide.md - 실제 사용법 가이드

> **네비게이션:** 목차 | 다음: 데이터 흐름 분석